

PROJET OSIRIS

Plateforme de supervision de sécurité SIEM / XDR open-source

Rapport final de projet

Du besoin client à la plateforme d'entreprise : conception, déploiement progressif et validation d'une architecture Wazuh en trois phases

Intitulé	Projet OSIRIS - plateforme de supervision SIEM / XDR open-source
Groupe	SR02 / SR-2E - EFREI Paris
Commanditaire	RSSI d'une PME (organisation cliente) · Sponsor : Direction générale
Membres	Yves de Kerros (Chef de projet / MOE) · Robinson Diallo (MOA) · Nathan Favry (Responsable technique) · Anaïs Djenadi (Gouvernance & Communication) · Noor Fadlane (Risques, Budget, Qualité & Documentation)
Date	6 juillet 2026
Statut	Rapport final

Référence technologique : Wazuh 4.14.5 · pfSense · Docker · Suricata · VMware

Résumé

Contexte / enjeux. Une PME cliente, représentée par son RSSI, ne dispose d'aucune supervision de sécurité centralisée : ses journaux d'événements (serveurs, postes, équipements réseau) restent dispersés et aucune capacité de détection des menaces n'est en place. L'organisation est donc aveugle face aux attaques (force brute, altération de fichiers, scans réseau) et peine à démontrer sa conformité réglementaire, à commencer par le RGPD. Le projet OSIRIS répond à ce besoin en concevant et en déployant une plateforme de supervision de sécurité SIEM & XDR reposant intégralement sur des technologies open-source, sans coût de licence.

Solution. Pour une PME qui a besoin d'une visibilité unifiée sur la sécurité de son système d'information, OSIRIS est une plateforme SIEM/XDR fondée sur Wazuh qui collecte, corrèle et détecte les événements de sécurité de sources hétérogènes (Windows, Linux, serveur web, pare-feu), surveille l'intégrité des fichiers et réagit automatiquement aux attaques. À la différence d'un simple collecteur de journaux, elle transforme des événements isolés en alertes qualifiées, cartographie les menaces sur le référentiel MITRE ATT&CK et embarque nativement des vues de conformité (RGPD, ISO/IEC 27001). Son architecture unique se décline sans changement d'outil du déploiement tout-en-un jusqu'au cluster conteneurisé de niveau entreprise.

Résultats. La plateforme a été construite selon une trajectoire en trois phases de maturité croissante, chacune validée par une recette : (1) une architecture tout-en-un supervisant quatre sources et couvrant les quatre fonctions exigées ; (2) une architecture distribuée à trois nœuds enrichie de corrélation personnalisée, de détection de vulnérabilités, d'alerting et de gestion du cycle de vie des logs ; (3) une architecture d'entreprise résiliente et conteneurisée (cluster indexer à réplication, cluster de managers avec équilibrage de charge HAProxy sous Docker), dotée d'un IDS réseau (Suricata), de renseignement sur les menaces (listes d'IoC + VirusTotal) et d'une réponse automatisée aux incidents. L'ensemble a été réalisé sous une contrainte matérielle forte, un hôte de virtualisation de 16 Go de RAM, grâce à une discipline stricte de groupes d'allumage et de snapshots.

Conclusions. OSIRIS démontre qu'une PME peut se doter d'une supervision de sécurité de niveau professionnel, évolutive et conforme au RGPD, à coût de licence nul. Le SIEM/XDR passe d'un outil qui observe à une plateforme qui détecte, corrèle et répond. Certaines exigences avancées de la phase 3 (Active Directory, cloud AWS, analyse comportementale étendue, conformité PCI DSS) n'ont pas été réalisées faute de ressources matérielles et de temps ; elles sont documentées de façon transparente en tant que travail restant et perspectives d'industrialisation.

Mots-clés

SIEM · XDR · Wazuh · supervision de sécurité · corrélation d'événements · détection de vulnérabilités · IDS Suricata · threat intelligence · Active Response · cluster · conteneurisation Docker · HAProxy · RGPD · ISO 27001 · MITRE ATT&CK · open-source · PME

Historique des révisions

Ce rapport final capitalise l'ensemble des livrables produits au fil du projet : documents de cadrage, rapports d'avancement et documents de réalisation de chacune des trois phases.

Version	Date	Auteur(s)	Évolutions
0.1	15 juin 2026	Groupe SR-2E	Cadrage, besoins client, état de l'art, choix de Wazuh
0.2	22 juin 2026	Groupe SR-2E	Phase 1 validée (tout-en-un, 4 sources, 4 fonctions, RGPD)
0.3	29 juin 2026	Groupe SR-2E	Phase 2 validée (architecture distribuée, corrélation, vulnérabilités, alerting, ISM, ISO 27001)
0.4	5 juillet 2026	Groupe SR-2E	Phase 3 (cluster HA, conteneurs, HAProxy, Suricata, threat intel, Active Response)
1.0	9 juillet 2026	Groupe SR-2E	Rapport final consolidé - version de soutenance

Table des matières

Le présent document est autonome : il contient l'ensemble des informations nécessaires à sa compréhension par un lecteur non spécialiste, notamment un responsable de PME. Un glossaire et une liste d'acronymes sont fournis en section 2. Les blocs de code apparaissent sur fond sombre en police à chasse fixe ; les encadrés colorés signalent les points d'attention, pièges et enseignements clés.

Résumé	1
Historique des révisions.....	2
Table des matières	3
1. Introduction	4
2. Documentation et terminologie.....	5
2.1 Documents de référence	6
2.2 Glossaire.....	6
2.2.1 Termes	6
2.2.2 Acronymes.....	6
3. Description du projet.....	7
3.1 Origine du projet	8
3.2 État de l'art et choix de la solution	8
3.2.1 L'infrastructure : virtualisation, pare-feu et systèmes.....	8
3.3 Description de la solution livrée	9
3.3.1 Les trois phases en synthèse	10
3.4 Caractère innovant et valeur ajoutée	10
3.5 Liberté d'exploitation et risques.....	10
3.6 Aspect réglementaire.....	11
3.7 Étude de marché, positionnement et modèle économique	11
3.7.1 Clients potentiels et quantification	11
3.7.2 Benchmark et avantages concurrentiels.....	11
3.7.3 Modèle économique et coût-bénéfice	12
3.7.4 Coût-bénéfice environnemental.....	12
3.8 Programme de travail et jalons	12
3.9 Devenir du produit et durée de vie	12
3.10 Positionnement et création de valeur pour une PME	12
4. Fonctions et contribution des acteurs	13
4.1 Contexte et organisation.....	14
4.2 Rôles et contributions.....	14
4.3 Méthodologie et calendrier	14
5. Études de faisabilité et environnement de test	14
5.1 Environnement physique	15
5.2 Environnement matériel	15
5.2.1 Groupes d'allumage (discipline des 16 Go)	15
5.3 Environnement logiciel	16

5.4 Données	16
6. Réalisation et mise en œuvre.....	16
6.1 Phase 1 - Déploiement tout-en-un	17
a) Réseau et pare-feu	17
b) Installation du serveur Wazuh	17
c) Déploiement des agents et source agentless.....	17
d) Fonctionnalités et résultats validés.....	17
6.2 Phase 2 - Architecture distribuée	18
a) Déploiement distribué	18
b) Corrélation personnalisée et processus suspect.....	18
c) Détection de vulnérabilités	18
d) Alerting (e-mail + webhook)	19
e) Rétention et conformité ISO 27001	20
6.3 Phase 3 - Architecture d'entreprise (cluster, conteneurs, IDS, threat intel, réponse)	20
a) Haute disponibilité : cluster indexer à deux nœuds.....	20
b) Cluster de managers et conteneurisation	20
c) Équilibrage de charge : HAProxy	21
d) IDS réseau : Suricata intégré à Wazuh.....	21
e) Threat intelligence : listes d'loC + VirusTotal	22
f) Réponse automatisée aux incidents (Active Response)	22
g) Vue d'ensemble de la plateforme	24
7. Suite de tests de recette	25
7.0 Scénarios nominaux détaillés.....	26
Scénario nominal A - Détection et corrélation d'une force brute SSH	26
Scénario nominal B - Threat intelligence : fichier malveillant détecté par VirusTotal	26
7.1 Exigence 1 - Collecte centralisée et supervision (Phase 1)	26
7.2 Exigence 2 - Détection et corrélation (Phases 1-2).....	27
7.3 Exigence 3 - Intégrité des fichiers (FIM, Phase 1)	27
7.4 Exigence 4 - Vulnérabilités, alerting, rétention, conformité (Phase 2).....	27
7.5 Exigence 5 - Cluster, conteneurs, IDS, threat intel, réponse (Phase 3).....	27
8. Périmètre non réalisé et travail restant.....	28
9. Conclusions et perspectives	29
9.1 Bilan et compétences acquises	30
9.2 Analyse critique et limites.....	30
9.3 Perspectives	30
Annexe A - Plan d'adressage et flux réseau	31
A.1 Plan d'adressage complet (VMnet3 - 192.168.50.0/24).....	32
A.2 Flux et ports principaux	32
Annexe B - Bonnes pratiques et pièges consolidés.....	32
Annexe C - Intitulés des sections (rapport en français).....	32

1. Introduction

La cybersécurité est devenue une fonction critique pour les organisations : multiplication des attaques, durcissement des obligations réglementaires (RGPD, ISO/IEC 27001, PCI DSS) et dépendance croissante au système d'information. Dans ce contexte, de nombreuses PME collectent des journaux d'événements sans jamais les exploiter, faute d'un outil centralisé. Elles restent alors aveugles face aux attaques et incapables de démontrer leur conformité. Le projet OSIRIS s'attaque directement à ce problème : offrir à une PME une visibilité unifiée sur la sécurité de son infrastructure et de réelles capacités de détection, tout en respectant le RGPD et en maîtrisant les coûts.

Description du système. OSIRIS est une plateforme de supervision de sécurité qui réunit deux briques complémentaires : un SIEM (Security Information and Event Management), qui centralise et corrèle les journaux de l'ensemble du parc, et un XDR (Extended Detection and Response), qui étend la détection jusqu'aux postes et serveurs au moyen d'agents et permet de réagir automatiquement aux incidents. La solution repose sur Wazuh, déployé sur une infrastructure virtualisée VMware protégée par un pare-feu pfSense. Elle collecte les événements de machines Windows, Linux, d'un serveur web et du pare-feu, les analyse au travers d'un moteur de règles, et restitue le tout dans des tableaux de bord unifiés.

Objectif de ce document. Ce rapport final présente l'intégralité de ce qui a été réalisé au cours du projet : les choix techniques et leur justification, la manière dont chaque brique a été mise en œuvre, les problèmes rencontrés et leurs solutions, et les résultats validés par des tests concrets. Il est rédigé pour permettre à un client ou à une PME de comprendre à cent pour cent la solution : comment elle fonctionne, comment l'utiliser et quelles garanties elle apporte. Les exigences non réalisées sont, elles aussi, clairement identifiées afin de ne présenter que le périmètre effectivement livré.

Comment lire ce rapport

Structure. La section 2 fixe le vocabulaire (glossaire, acronymes). La section 3 décrit le projet, son état de l'art et la solution. La section 4 présente les acteurs et la méthodologie. La section 5 détaille l'environnement de test (physique, matériel, logiciel, données). La section 6 expose la réalisation et la mise en œuvre des trois phases. La section 7 rassemble la suite de tests de recette. La section 8 recense honnêtement le périmètre non réalisé. La section 9 conclut et ouvre des perspectives.

Conventions. Les adresses du laboratoire sont en 192.168.50.0/24. Les commandes et extraits de configuration sont présentés en blocs de code. Les encadrés bleus donnent des explications, les oranges des pièges, les verts des validations.

2. Documentation et terminologie

2.1 Documents de référence

Document	Référence	Joint ?	Rôle vis-à-vis du projet
Sujet - SIEM & XDR Deployment Project	EFREI, avril 2026	Oui	Cahier des charges du projet (exigences des trois phases)
Besoins du client et objectifs	Groupe SR02	Oui	Synthèse du besoin et des exigences de conformité
Solution envisagée & état de l'art	Groupe SR02	Oui	Justification du choix de Wazuh et de l'infrastructure
Réalisation & rapports d'avancement (Ph. 1-2-3)	Groupe SR02	Oui	Détail technique de chaque phase (source de ce rapport)
Rapports de gestion de projet (TD1, TD2)	Groupe SR-2E	Oui	Cadrage, planification, cycle en V, risques
Documentation Wazuh 4.14	wazuh.com/docs	Non	Référence produit (installation, règles, modules)

2.2 Glossaire

2.2.1 Termes

Terme	Définition
Agent	Petit logiciel installé sur une machine surveillée (endpoint) qui collecte ses journaux et les remonte, chiffrés, vers le serveur d'analyse.
Agentless	Mode de collecte sans agent : un équipement (ex. pare-feu) envoie ses journaux par le protocole syslog.
Corrélation	Regroupement de plusieurs événements liés pour faire émerger une alerte de plus haut niveau (ex. 8 échecs SSH = une force brute).
FIM	File Integrity Monitoring : surveillance de l'intégrité des fichiers sensibles ; toute création, modification ou suppression déclenche une alerte.
Cluster	Ensemble de plusieurs serveurs coopérant comme un seul, pour la performance et la tolérance aux pannes.
Réplication	Duplication des données sur plusieurs nœuds afin qu'aucune ne soit perdue si un serveur tombe.
Conteneur	Unité logicielle isolée et légère (Docker) embarquant une application et ses dépendances.
Équilibrage de charge	Répartition automatique du trafic entre plusieurs serveurs (ici via HAProxy).
Threat intelligence	Renseignement sur les menaces : confrontation de l'activité observée à des indicateurs de compromission connus (IP, hachages).
Active Response	Mécanisme par lequel la plateforme agit automatiquement en réaction à une alerte (ex. blocage d'une IP attaquante).
Snapshot	Photographie de l'état d'une machine virtuelle permettant d'y revenir en cas d'erreur.

2.2.2 Acronymes

Acronyme	Signification	Explication
SIEM	Security Information and Event Management	Centralisation et corrélation des journaux de sécurité.
XDR	Extended Detection and Response	Détection et réponse étendues jusqu'aux endpoints.
IDS / NIDS	(Network) Intrusion Detection System	Détection d'intrusion par analyse du trafic réseau (Suricata).

Acronyme	Signification	Explication
HIDS	Host Intrusion Detection System	Détection basée sur les journaux des hôtes (cœur de Wazuh).
CVE	Common Vulnerabilities and Exposures	Identifiant standard d'une vulnérabilité logicielle connue.
IoC	Indicator of Compromise	Marqueur d'une compromission (IP, hachage, domaine malveillant).
ISM	Index State Management	Gestion du cycle de vie des index (rétention automatique).
FIM	File Integrity Monitoring	Surveillance de l'intégrité des fichiers.
RGPD	Règlement Général sur la Protection des Données	Cadre européen de protection des données personnelles.
MITRE ATT&CK	Adversarial Tactics, Techniques and Common Knowledge	Référentiel des tactiques et techniques d'attaquants.
MOA / MOE	Maîtrise d'Ouvrage / Maîtrise d'Œuvre	Le commanditaire (besoin) et le réalisateur (solution).
HA	High Availability	Haute disponibilité : maintien du service malgré une panne.

3. Description du projet

Cette section décrit l'origine du projet, l'état de l'art des solutions disponibles, la solution retenue et sa mise en œuvre, son caractère innovant, le cadre réglementaire et le modèle de valeur pour une PME. Elle constitue le cœur du rapport et donne au lecteur une compréhension complète de ce qui a été conçu et livré.

3.1 Origine du projet

Le projet naît d'un constat simple et fréquent chez les PME : les journaux d'événements existent (serveurs, postes, pare-feu) mais ne sont jamais exploités de façon centralisée. Sans corrélation ni tableau de bord, l'organisation ne voit pas les attaques et ne peut pas prouver sa conformité. Le RSSI de la PME cliente (commanditaire), soutenu par la Direction générale (sponsor), a donc demandé la conception d'une plateforme de supervision de sécurité open-source, maîtrisable en interne et sans coût de licence, capable de grandir d'un déploiement simple vers une solution d'entreprise.

Le sujet impose une démarche progressive en trois niveaux de maturité, tout-en-un, distribué, puis conteneurisé, et une conformité qui se renforce à chaque étape (RGPD dès le départ, puis ISO/IEC 27001, puis PCI DSS). Il laisse volontairement le choix de l'outil aux étudiants, la première tâche étant d'analyser, évaluer et sélectionner l'architecture open-source la plus adaptée.

Le besoin en une phrase

Le client attend une plateforme de supervision open-source, centralisée et conforme au RGPD, capable d'intégrer des sources variées et d'évoluer du simple poste tout-en-un vers une solution distribuée et automatisée de niveau entreprise.

3.2 État de l'art et choix de la solution

Le monde open-source propose plusieurs briques de supervision, mais peu réunissent à la fois le SIEM (collecte et corrélation de logs) et le XDR (détection sur les endpoints via un agent). Nous avons comparé les principales solutions selon quatre critères issus du besoin : couverture fonctionnelle, gratuité et maîtrise interne, capacité d'évolution, et simplicité de mise en œuvre en laboratoire.

Solution	Type	Forces	Limite pour ce projet
Wazuh	SIEM + XDR	FIM, agents multi-OS + agentless, MITRE ATT&CK et RGPD intégrés, du tout-en-un au cluster	Retenu - couvre l'ensemble du besoin
Security Onion	SIEM + NIDS	Détection réseau très forte (Suricata, Zeek)	Lourd, orienté réseau ; volet endpoint moins central
Elastic (ELK) + Elastic Security	SIEM / analytics	Très puissant et scalable	FIM et agentless à monter à la main ; plus complexe
Graylog	Gestion de logs	Excellent moteur de collecte et de recherche	Pas de XDR ni de FIM natifs ; détection limitée
OSSIM (AlienVault)	SIEM	Solution tout-en-un historique	Projet vieillissant, communauté en déclin

Solution retenue : Wazuh. Wazuh couvre nativement les quatre fonctions exigées en phase 1 (collecte centralisée, détection d'événements, surveillance d'intégrité des fichiers, tableau de bord). Il gère aussi bien les agents (Windows, Linux) que l'agentless par syslog (pare-feu, switch), et embarque déjà le mapping MITRE ATT&CK et un module de conformité RGPD. L'argument décisif est que son architecture native suit exactement la progression du projet, déploiement tout-en-un, puis multi-nœuds, puis cluster conteneurisé : on ne change jamais d'outil entre les phases. Wazuh s'articule autour de quatre briques : le manager (analyse et règles), l'indexer (stockage et recherche), le dashboard (visualisation) et l'agent (capteur installé sur les hôtes).

3.2.1 L'infrastructure : virtualisation, pare-feu et systèmes

L'ensemble est déployé sur des machines virtuelles, afin de reproduire une infrastructure d'entreprise hétérogène à moindre coût et de pouvoir revenir en arrière (snapshots) en cas d'erreur. Les choix d'infrastructure et leur justification :

Composant	Choix	Justification principale
Hyperviseur	VMware Workstation Pro	Déjà maîtrisé (TP), gestion fine des réseaux virtuels (VMnet)
Pare-feu / routeur	pfSense CE	Référence open-source, reconnu nativement par les décodeurs Wazuh
Serveur SIEM & endpoints Linux	Ubuntu Server 24.04 LTS	Version LTS officiellement supportée par Wazuh, sans interface graphique
Poste de travail	Windows 10 Pro	Source d'événements hétérogène réaliste (journaux Windows)
Machine d'attaque	Kali Linux	Outils de test et d'attaque pour valider la détection
Réseau du projet	VMnet3 (192.168.50.0/24)	Bulle réseau isolée des autres travaux pratiques

3.3 Description de la solution livrée

La solution finale est une plateforme SIEM/XDR distribuée et conteneurisée, construite en trois phases dont chacune enrichit la précédente sans jamais la remettre en cause. Le laboratoire reproduit une PME réaliste : un pare-feu en passerelle, un serveur de supervision, des endpoints hétérogènes et une machine d'attaque, le tout sur un réseau isolé.

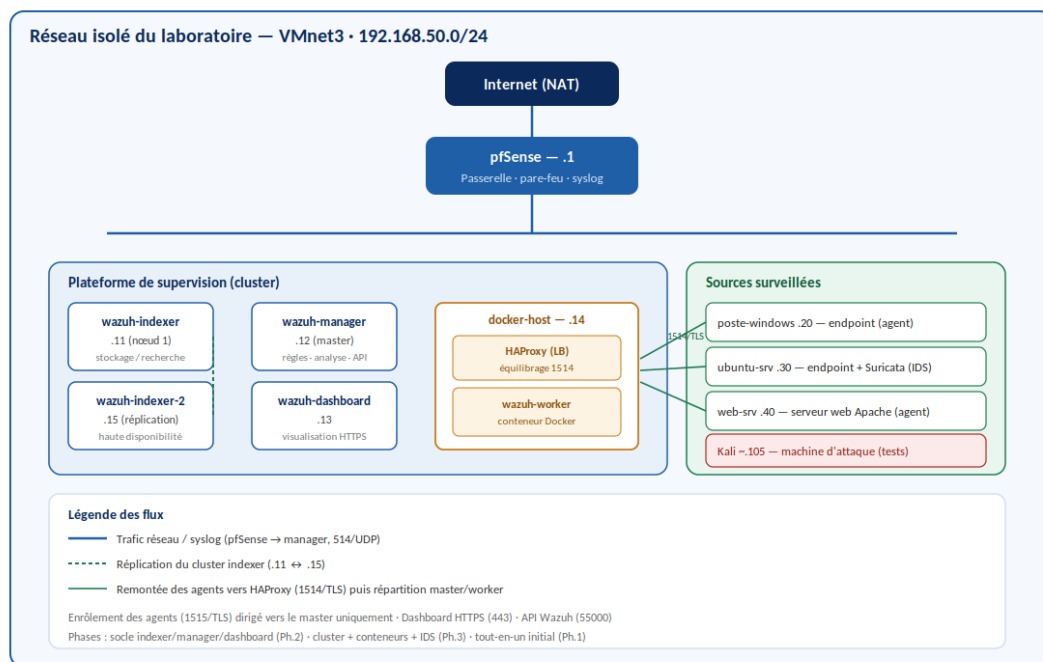


Figure 1 - Architecture cible d'OSIRIS sur le réseau isolé VMnet3 (192.168.50.0/24) : pfSense en passerelle, le cluster indexer à réplication, le master et le worker conteneurisé derrière HAProxy, le dashboard, les endpoints supervisés et la machine d'attaque Kali.

Fonctionnement d'ensemble. Chaque machine surveillée exécute un agent Wazuh qui remonte ses événements chiffrés (TLS) vers le manager ; le pare-feu, lui, est intégré sans agent via syslog. Le manager décode et analyse chaque événement au travers d'un pipeline, pré-décodage, décodage des champs métier, application de règles générant un niveau de gravité (0 à 15), puis indexation. Le dashboard restitue le tout : alertes par gravité, inventaire des agents, vues MITRE ATT&CK, vulnérabilités et conformité. La valeur ajoutée d'un SIEM tient dans la **corrélation** : un événement isolé (un échec SSH, une erreur 404) est peu inquiétant, mais plusieurs événements rapprochés d'une même source déclenchent une alerte de haut niveau.

Comment l'utiliser. L'analyste ou l'administrateur accède à la plateforme via un navigateur (interface HTTPS). Il y consulte les tableaux de bord, recherche des événements, examine les alertes et leurs étiquettes de conformité, et reçoit des notifications (e-mail, Discord) pour les alertes critiques. Les règles de détection et la configuration se gèrent côté manager ; le déploiement d'un nouvel agent s'effectue en quelques commandes depuis le dashboard. L'exploitation quotidienne (surveillance des alertes, mises à jour, rotation des journaux) relève ensuite de l'opération courante, distincte du présent projet de conception.

3.3.1 Les trois phases en synthèse

Phase	Architecture	Apports principaux	Conformité
1 - Minimale	Tout-en-un	4 sources (Windows, Linux, web, pare-feu), collecte, détection, FIM, dashboard	RGPD
2 - Intermédiaire	Distribuée (3 nœuds)	Corrélation personnalisée, vulnérabilités (CVE), alerting, rétention (ISM)	RGPD + ISO 27001
3 - Avancée	Cluster + conteneurs	HA indexer, worker Docker + HAProxy, Suricata (IDS), threat intel, Active Response	RGPD + ISO 27001

Le détail de la mise en œuvre de chaque phase, commandes, configurations, problèmes rencontrés et résultats, est présenté en section 6 (réalisation et mise en œuvre) et validé en section 7 (suite de tests).

3.4 Caractère innovant et valeur ajoutée

L'innovation d'OSIRIS n'est pas technologique au sens d'une invention, mais architecturale et méthodologique : démontrer qu'une PME peut se doter, sans budget de licence, d'une chaîne SIEM/XDR complète et évolutive, en franchissant proprement les paliers de maturité d'une véritable infrastructure d'entreprise.

- **Une solution unique sur les trois phases.** Le même outil (Wazuh) porte le tout-en-un, le distribué et le cluster conteneurisé. On capitalise à chaque étape au lieu de repartir de zéro, un avantage décisif de maintenabilité et de coût de formation.
- **De l'observation à la réponse.** La plateforme ne se contente pas de détecter : elle réagit. La corrélation transforme le bruit en alertes qualifiées, et l'Active Response bloque automatiquement une IP attaquante. C'est le passage concret du SIEM (qui observe) au XDR (qui répond).
- **Frugalité assumée.** Toute l'architecture d'entreprise (cluster à réplication, conteneurs, équilibrage de charge, IDS) tient sur un hôte de 16 Go grâce à des choix économes et une discipline stricte de groupes d'allumage. La démonstration reste pleinement valable tout en restant reproductible par n'importe quelle PME.

Progrès par rapport à l'existant

Beaucoup d'organisations en restent à la simple journalisation. OSIRIS montre le saut qualitatif vers la corrélation (relier des événements), l'enrichissement (threat intelligence, CVE, MITRE ATT&CK) et la réponse automatisée, trois capacités habituellement réservées à des SOC dotés d'outils propriétaires coûteux.

3.5 Liberté d'exploitation et risques

Le projet ne présente pas de risque de contrefaçon : tous les composants sont open-source et utilisés conformément à leurs licences respectives (Wazuh sous licence GPLv2 pour l'agent et Apache 2.0 pour la partie serveur, pfSense sous licence Apache 2.0, Ubuntu, Suricata sous GPLv2, HAProxy sous GPLv2). Aucune dépendance propriétaire n'est intégrée à la chaîne livrée. La matrice ci-dessous récapitule les principaux risques du projet et leur maîtrise.

Risque	Nature	Criticité	Mesure de maîtrise
Saturation mémoire de l'hôte	Technique	Élevée	Groupes d'allumage, snapshots, distribution des composants

Risque	Nature	Criticité	Mesure de maîtrise
Glissement de périmètre	Organisationnel	Moyenne	Périmètre figé par phase, arbitrage en comité
Retard de déploiement	Planning	Moyenne	Marges au planning, re-séquencement, report d'options
Non-conformité RGPD	Réglementaire	Faible mais critique	Module GDPR, TLS/HTTPS, RBAC, rétention
Faux positifs de détection	Qualité	Moyenne	Réglage fin des règles, seuils de corrélation, tests
Dépendance licence / contrefaçon	Juridique	Très faible	100 % open-source, respect des licences

3.6 Aspect réglementaire

En France et en Europe. La plateforme traite des journaux qui contiennent des données personnelles (adresses IP, identifiants) : le SIEM est lui-même un traitement à sécuriser au sens du RGPD. La conformité a donc été intégrée dès la conception (privacy by design).

Principe RGPD	Mise en œuvre dans OSIRIS
Sécurité du traitement (art. 32)	TLS entre agent et manager, HTTPS pour le dashboard, certificats générés à l'installation
Limitation de conservation (art. 5)	Politique de rétention automatique (ISM) et rotation des journaux
Minimisation (art. 5)	Collecte des seuls journaux utiles ; exclusions sur le FIM
Contrôle d'accès (art. 32)	RBAC du dashboard : comptes nominatifs, rôles limités
Intégrité (art. 5)	Le FIM détecte toute altération non autorisée des fichiers

Wazuh mappe automatiquement ses règles aux articles du RGPD (étiquettes de type `gdpr_IV_32.2` sur les alertes) et fournit un module de conformité dédié. **À l'international**, la même mécanique d'étiquetage couvre l'ISO/IEC 27001:2022 (traitee en phase 2 via l'évaluation de configuration SCA/CIS et des tags de contrôle), et les référentiels HIPAA, NIST 800-53 et PCI DSS sont pré-étiquetés par le jeu de règles, ce qui facilite une extension ultérieure de la conformité.

3.7 Étude de marché, positionnement et modèle économique

Le marché de la supervision de sécurité est dominé par des SIEM propriétaires (Splunk, IBM QRadar, Microsoft Sentinel) au coût de licence élevé, souvent facturés au volume de données ingérées et hors de portée d'une PME. Face à eux, un écosystème open-source mûr s'est développé, dont Wazuh est l'un des leaders (plusieurs millions de téléchargements annuels, communauté active). OSIRIS se positionne précisément sur ce segment open-source, maîtrisable en interne et à coût de licence nul, qui correspond au besoin d'une PME cherchant une visibilité de sécurité sans engager un budget d'outillage disproportionné.

3.7.1 Clients potentiels et quantification

La cible première est la PME et l'ETI disposant d'un système d'information hétérogène (postes, serveurs, équipements réseau) mais sans SOC externalisé ni budget SIEM propriétaire. En France, cela représente un vivier très large : la grande majorité des PME dispose d'une infrastructure informatique à superviser, et le durcissement réglementaire (RGPD, directive NIS2 pour les entités essentielles et importantes) pousse ces organisations à se doter d'une capacité de détection et de traçabilité. Les clients secondaires sont les prestataires de services managés (MSP) souhaitant proposer une supervision mutualisée à leurs clients, et les établissements de formation en cybersécurité.

3.7.2 Benchmark et avantages concurrentiels

Critère	SIEM propriétaire	OSIRIS (Wazuh)
Coût de licence	Élevé, souvent au volume	Nul (open-source)
Maîtrise interne	Dépendance à l'éditeur	Totale (code ouvert, config locale)
SIEM + XDR + FIM	Selon modules payants	Natif et intégré
Évolutivité	Oui, mais coûteuse	Native (tout-en-un → cluster) sans surcoût
Conformité clé en main	Oui (souvent en option)	RGPD/ISO/PCI DSS pré-étiquetés
Courbe d'apprentissage	Formation éditeur	Documentation communautaire abondante

3.7.3 Modèle économique et coût-bénéfice

Le modèle repose sur le coût total de possession plutôt que sur la licence : l'investissement principal est le temps de mise en place et d'exploitation, l'infrastructure étant virtualisée et réutilisée. Pour la PME cliente, le retour sur investissement tient à la réduction du risque (détection et réaction plus rapides aux incidents, qui coûtent cher lorsqu'ils ne sont pas contenus) et à la conformité démontrable (utile en cas d'audit ou de contrôle de la CNIL). Le budget réellement engagé sur le projet est nul (plafond de sécurité de 50 € pour d'éventuels imprévus, non consommé). Un modèle de revenus est envisageable pour un prestataire : facturation d'une prestation d'intégration et d'un support/maintien en condition opérationnelle, la plateforme elle-même restant gratuite.

3.7.4 Coût-bénéfice environnemental

La frugalité de la solution a un bénéfice environnemental direct : l'ensemble d'une architecture d'entreprise tient sur un seul hôte de 16 Go grâce à des composants légers (Samba plutôt que Windows Server, Docker plutôt que Kubernetes, groupes d'allumage). Cette sobriété matérielle réduit la consommation électrique et l'empreinte carbone par rapport à un déploiement propriétaire nécessitant des serveurs dédiés surdimensionnés. En production, la mutualisation des ressources (conteneurs, virtualisation) prolonge cet avantage.

3.8 Programme de travail et jalons

Le projet a été conduit selon un programme en trois phases, chacune bornée par un jalon de recette. Le tableau ci-dessous récapitule les jalons techniques et, pour information, les jalons « marché » et « sécurité juridique » (propriété intellectuelle) qui structureraient une industrialisation.

Jalon	Type	Contenu	Statut
J1 - Recette Phase 1	Technique	Tout-en-un : 4 sources, 4 fonctions, RGPD	✓ Validé
J2 - Recette Phase 2	Technique	Distribué : corrélation, CVE, alerting, ISO 27001	✓ Validé
J3 - Recette Phase 3	Technique	Cluster HA, conteneurs, HAProxy, IDS, threat intel	✓ Validé
Mise en exploitation	Marché	Transfert au SOC/administrateur de la PME	🟡 Intégré
Sécurité juridique (PI)	Juridique	Vérification des licences open-source utilisées	✓ Validé

Sécurité juridique (propriété intellectuelle). Tous les composants sont utilisés conformément à leurs licences open-source respectives (GPLv2, Apache 2.0). Aucune brique propriétaire n'est intégrée ; le risque de contrefaçon est donc nul et la liberté d'exploitation totale (cf. §3.5).

3.9 Devenir du produit et durée de vie

À l'issue du projet, la plateforme est transférée à l'exploitation de la PME (SOC ou administrateur système) : la conception s'achève, l'opération courante commence (surveillance des alertes, mises à jour, ajustement des règles et de la rétention). La durée de vie du produit est celle de l'infrastructure supervisée ; l'architecture en phases garantit qu'il pourra évoluer progressivement, ajout de nœuds indexer, de workers, d'intégrations cloud ou d'un IDS supplémentaire, au rythme des besoins et du budget, sans jamais changer d'outil. À plus long terme, la plateforme peut servir de socle à un service de supervision mutualisé (MSP) ou être migrée vers une orchestration Kubernetes pour un passage à l'échelle.

3.10 Positionnement et création de valeur pour une PME

Le marché de la supervision de sécurité est dominé par des SIEM propriétaires (Splunk, IBM QRadar, Microsoft Sentinel) au coût de licence élevé, souvent hors de portée d'une PME. OSIRIS se positionne sur le segment open-source, maîtrisable en interne, dont Wazuh est l'un des leaders. La création de valeur pour la PME cliente est directe :

- **Coût.** Zéro licence, infrastructure virtualisée réutilisée : le principal investissement est le temps de mise en place, ensuite amorti par l'exploitation interne.
- **Réduction du risque.** Détection et corrélation temps réel, surveillance d'intégrité, réponse automatisée : le temps de détection et de réaction aux incidents diminue fortement.
- **Conformité démontrable.** Vues et rapports RGPD/ISO 27001 clé en main, utiles en cas d'audit ou de contrôle.
- **Évolutivité.** La même plateforme accompagne la croissance de la PME, du poste unique au cluster, sans changement d'outil ni de compétences.

Cycle de vie du produit

À l'issue du projet, la plateforme est transférée à l'exploitation (SOC / administrateur de la PME) : surveillance des alertes, mises à jour, ajustement des règles et de la rétention. Sa durée de vie est celle de l'infrastructure supervisée ; l'architecture en phases permet de la faire évoluer progressivement (ajout de nœuds, de workers, d'intégrations cloud) au rythme des besoins et du budget.

4. Fonctions et contribution des acteurs

4.1 Contexte et organisation

Le projet a été mené par l'équipe OSIRIS (groupe SR-2E), équipe temporaire et pluridisciplinaire mandatée par le RSSI d'une PME cliente (commanditaire / MOA), avec la Direction générale comme sponsor. L'organisation retenue est matricielle : chaque membre porte une expertise spécialisée (besoins, technique, gouvernance, risques, documentation) tout en contribuant à l'objectif commun sous la coordination du chef de projet.

4.2 Rôles et contributions

Membre	Rôle	Contribution principale
Yves de Kerros	Chef de projet / MOE	Coordination, planning, cohérence du dossier et de la soutenance
Robinson Diallo	Responsable MOA	Analyse du besoin, objectifs, cahier des charges fonctionnel
Nathan Favry	Responsable technique	Déploiement Wazuh, choix techniques, infrastructure, livrables
Anaïs Djenadi	Gouvernance & Communication	Parties prenantes, instances de pilotage, plan de communication
Noor Fadlane	Risques, Budget, Qualité & Doc.	Registre des risques, budget, indicateurs, mise en forme

4.3 Méthodologie et calendrier

La démarche est hybride : une ossature prédictive (trois phases bornées, chacune validée par une recette à son jalon, le squelette d'un cycle en V) combinée à une conduite itérative de chaque phase (on déploie, on teste, on ajuste, on capitalise avant de passer à la suivante). Cette hybridation sécurise les jalons de validation exigés par le commanditaire tout en gardant la souplesse nécessaire face aux incertitudes techniques.

Le suivi s'appuie sur des réunions d'équipe hebdomadaires, un suivi des tâches partagé, et une revue de jalon (recette + validation) à la fin de chaque phase. Le projet s'est déroulé sur environ cinq semaines, de début juin à la soutenance du 9 juillet 2026.

Jalon	Échéance	Contenu validé
J0 - Cadrage	Fin S1	Besoin, objectifs et périmètre approuvés
J1 - Recette Phase 1	Fin S2	Tout-en-un : 4 sources, 4 fonctions, RGPD
J2 - Recette Phase 2	Fin S3	Architecture distribuée, corrélation, alerting, ISO 27001
J3 - Recette Phase 3	Fin S4	Cluster HA, conteneurs, Suricata, threat intel, Active Response
J4 - Soutenance	9 juillet 2026	Présentation du dossier complet au comité de pilotage

Outils de travail, de code et de documentation

Infrastructure : VMware Workstation Pro, pfSense, Ubuntu Server, Docker. **Diagnostic** : wazuh-logtest, wazuh-analysisd -t, cluster_control, agent_control, ss, tcpdump, jq. **Alerting** : webhook Discord, relais Postfix vers Gmail. **Documentation & suivi** : rapports d'avancement et de réalisation par phase, suivi de tâches partagé. La reprise du projet est facilitée par des livrables reproductibles (commandes exactes) et des snapshots pris avant chaque opération sensible.

5. Études de faisabilité et environnement de test

Cette section décrit la configuration et l'installation de tous les éléments nécessaires à la réalisation et aux tests du projet, en partant d'une base vierge. Elle couvre l'environnement physique, matériel, logiciel et les données. La mise en œuvre effective des trois phases, commandes, problèmes rencontrés et solutions, fait l'objet de la section 6.

5.1 Environnement physique

Le projet est intégralement virtualisé : aucune infrastructure physique dédiée n'est nécessaire au-delà d'un poste de travail hôte. L'ensemble s'exécute sur un unique ordinateur portant l'hyperviseur, sur lequel sont créées toutes les machines virtuelles. Cet hôte doit disposer d'un accès Internet (pour télécharger les paquets Wazuh et les flux de vulnérabilités et d'loC) et de la virtualisation matérielle activée. Le réseau du laboratoire est entièrement contenu dans un réseau virtuel isolé, sans exposition vers l'extérieur autre que la sortie Internet contrôlée par le pare-feu.

Contrainte structurante : l'hôte 16 Go

Toute la faisabilité repose sur une contrainte forte : un hôte de virtualisation limité à 16 Go de RAM. L'indexer Wazuh (moteur Java) est gourmand. La règle absolue adoptée est de ne jamais démarrer toutes les machines simultanément : on maintient un socle permanent (pare-feu + serveurs de supervision) et on n'allume les endpoints qu'au moment des tests. Deux incidents de saturation mémoire en début de projet ont confirmé la nécessité de cette discipline, complétée par des snapshots systématiques.

5.2 Environnement matériel

Aucun matériel spécifique n'est requis hormis l'hôte de virtualisation. Les machines virtuelles ci-dessous constituent l'environnement de test ; leur dimensionnement a été calibré pour tenir dans les 16 Go de l'hôte en respectant les groupes d'allumage.

Machine virtuelle	Rôle	RAM	Réseau
pfSense	Passerelle, pare-feu, source syslog	1 Go	WAN=NAT, LAN=VMnet3
wazuh-indexer (.11)	Indexation / stockage (nœud 1)	4 Go	VMnet3
wazuh-indexer-2 (.15)	Indexation / stockage (nœud 2, HA)	2,5 Go	VMnet3
wazuh-manager (.12)	Analyse, règles, alertes, API (master)	3 Go	VMnet3
wazuh-dashboard (.13)	Visualisation HTTPS	2 Go	VMnet3
docker-host (.14)	Worker Wazuh + HAProxy (conteneurs)	2 Go	VMnet3
ubuntu-srv (.30)	Endpoint Linux + Suricata (IDS)	2 Go	VMnet3
web-srv (.40)	Serveur web Apache surveillé	2 Go	VMnet3
poste-windows (.20)	Endpoint Windows surveillé	4 Go	VMnet3
Kali (~.105)	Machine d'attaque (tests)	4 Go	VMnet3

5.2.1 Groupes d'allumage (discipline des 16 Go)

Chaque scénario de test précise les machines à démarrer et celles à éteindre. On ne cumule jamais deux groupes gourmands simultanément.

Groupe	Machines allumées	RAM approx.	Sert à démontrer
Socle	pfSense + indexer + manager + dashboard	~9-10 Go	Plateforme de base opérationnelle
Cluster / LB	Socle + docker-host + indexer-2	~14-15 Go	HA, worker conteneurisé, HAProxy, réplication

Groupe	Machines allumées	RAM approx.	Sert à démontrer
Détection	Socle + ubuntu-srv + Kali	~13 Go	Suricata, Active Response, threat intel, force brute

5.3 Environnement logiciel

Tous les logiciels de l'environnement de test sont open-source et gratuits. Les références de version utilisées sont les suivantes.

Logiciel	Version	Rôle
VMware Workstation Pro	-	Hyperviseur et gestion des réseaux virtuels
pfSense CE	2.8.0-RELEASE	Pare-feu / passerelle / source syslog
Ubuntu Server	24.04 LTS	OS du serveur SIEM et des endpoints Linux
Windows	10 Pro	OS du poste de travail surveillé
Wazuh	4.14.5	SIEM/XDR (indexer, manager, dashboard, agent)
Docker + Compose	-	Conteneurisation du worker et de HAProxy
HAProxy	2.9-alpine	Équilibrage de charge des managers
Suricata	8.0.5 (PPA OISF)	IDS réseau
Kali Linux	-	Outils de test et d'attaque (nmap, hydra, gobuster)

Wazuh repose lui-même sur OpenSearch (moteur de recherche et de stockage, dérivé d'Elasticsearch) pour l'indexer, et sur Filebeat pour l'acheminement des alertes du manager vers l'indexer. Ces composants sont installés et câblés automatiquement par l'assistant officiel de Wazuh.

5.4 Données

Les données manipulées sont exclusivement des journaux d'événements de sécurité, générés en continu par les machines du laboratoire et par les tests d'attaque. Il n'y a pas de jeu de données externe à fournir : les données de test sont produites à la demande (échecs de connexion SSH, modifications de fichiers, scans réseau, dépôt d'un fichier de test antivirus).

- **Nature.** Journaux système et applicatifs (SSH, Windows Event Log, Apache, auditd), journaux du pare-feu (syslog), alertes de l'IDS Suricata (format EVE JSON), inventaire logiciel (pour la détection de vulnérabilités).
- **Structure.** Après décodage, chaque événement devient une alerte structurée (JSON) portant un identifiant de règle, un niveau de gravité (0-15), l'agent source, des champs métier (IP, utilisateur, fichier) et des étiquettes de conformité (RGPD, ISO 27001, PCI DSS, MITRE ATT&CK).
- **Volumétrie.** De l'ordre de plusieurs centaines à quelques milliers d'événements par période de test, stockés dans des index quotidiens (wazuh-alerts-*). La rétention est gérée automatiquement (politique ISM, 90 jours par défaut) pour respecter la limitation de conservation.
- **Sensibilité.** Les journaux contiennent des données personnelles (adresses IP, identifiants). Ils sont chiffrés en transit (TLS) et protégés par le contrôle d'accès du dashboard, conformément au RGPD.

6. Réalisation et mise en œuvre

Cette section détaille comment la plateforme a été effectivement construite, phase par phase : les étapes clés, les commandes réellement employées, les problèmes rencontrés et la façon dont ils ont été résolus. Elle constitue la trace reproductible du travail réalisé et s'appuie sur l'environnement décrit en section 5. Les résultats obtenus sont ensuite formellement validés par la suite de tests de la section 7.

6.1 Phase 1 - Déploiement tout-en-un

La phase 1 pose les fondations : un réseau dédié isolé, le pare-feu en passerelle, un serveur Wazuh « trois-en-un » (manager + indexer + dashboard sur une seule machine), trois agents et le pare-feu intégré en syslog.

a) Réseau et pare-feu

Un réseau virtuel dédié VMnet3 (Host-Only, 192.168.50.0/24) est créé, avec le DHCP VMware désactivé : c'est pfSense qui distribue les adresses. Le pare-feu, cloné depuis un TP antérieur et rebasculé sur VMnet3, sert de passerelle vers Internet (via son interface WAN en NAT) et de source de journaux syslog.

b) Installation du serveur Wazuh

Sur une VM Ubuntu Server 24.04 à IP statique (192.168.50.10), l'installation tout-en-un se fait via l'assistant officiel, qui génère l'autorité de certification interne, installe et câble l'indexer, le manager, Filebeat et le dashboard :

```
curl -sO https://packages.wazuh.com/4.14/wazuh-install.sh
sudo bash ./wazuh-install.sh -a

# Récupération du mot de passe admin généré
sudo tar -O -xvf wazuh-install-files.tar \
  wazuh-install-files/wazuh-passwords.txt
```

Les quatre services (wazuh-indexer, wazuh-manager, wazuh-dashboard, filebeat) doivent être actifs. Le dashboard est ensuite accessible en HTTPS sur <https://192.168.50.10> (certificat auto-signé, avertissement normal).

c) Déploiement des agents et source agentless

Sur chaque endpoint Linux, l'agent s'installe en pointant le manager, puis s'enrôle automatiquement (il contacte le manager sur le port 1515, reçoit sa clé, puis remonte ses événements chiffrés sur le port 1514) :

```
sudo WAZUH_MANAGER="192.168.50.10" WAZUH_AGENT_NAME="ubuntu-srv" \
  apt install -y wazuh-agent
sudo systemctl enable --now wazuh-agent
```

Le poste Windows reçoit l'agent via le paquet MSI. Le pare-feu, lui, est intégré sans agent : on active la réception syslog côté manager (bloc <remote> sur le port 514/UDP) et on configure pfSense pour émettre ses journaux vers 192.168.50.10. La chaîne agentless a été vérifiée par capture réseau (tcpdump sur le port 514).

ID	Name	IP address	Group(s)	Operating system	Cluster node	Version	Status	Actions
001	poste-windows	192.168.50.20	default	Microsoft Windows 10 Pro 10.0.19045.6456	node01	v4.14.5	connected	ⓘ ⚙ ⋮
002	web-srv	192.168.50.40	default	Ubuntu 24.04.4 LTS	node01	v4.14.5	connected	ⓘ ⚙ ⋮

Figure 2 - Inventaire des agents enrôlés dans la console Wazuh (poste Windows et serveur Ubuntu, version 4.14.5), preuve de la prise en charge des sources hétérogènes.

d) Fonctionnalités et résultats validés

Les quatre fonctions exigées ont été éprouvées par des tests concrets : collecte centralisée (629 événements agrégés au dashboard, dont 356 de gravité moyenne et 273 faibles), détection de force brute SSH (règle native 5710 répétée puis règle 5712 de niveau 10 par corrélation), surveillance d'intégrité des fichiers (règle 553 à la suppression d'un fichier de /etc, avec étiquettes RGPD), et tableau de bord de synthèse. En bonus, un scan web (gobuster depuis Kali) a été détecté (règles 31101 puis 31151).

Enseignements de la Phase 1

RAM 16 Go - contrainte majeure. Le serveur réclame 8 Go ; un démarrage de trop de VM a provoqué un plantage mémoire. D'où la règle des groupes d'allumage.

Copier-coller console non fiable. Les commandes multi-lignes se cassent dans la console VMware : on privilégie une seule ligne ou une session SSH.

Où se lisent les alertes. Les alertes se lisent uniquement sur le serveur (fichier alerts.log) ; l'agent provoque l'événement, le manager le decode. Les logs du pare-feu (agentless) apparaissent rattachés au manager (agent 000).

6.2 Phase 2 - Architecture distribuée

La phase 2 « éclate » le tout-en-un en trois nœuds dédiés (indexer (.11), manager (.12), dashboard (.13)) puis enrichit la détection. Distribuer permet de dimensionner chaque nœud séparément, de fiabiliser l'ensemble (le redémarrage du dashboard n'affecte pas l'indexation) et d'ouvrir la voie au cluster de la phase 3.

a) Déploiement distribué

La méthode officielle repose sur un fichier config.yml décrivant les trois nœuds ; l'assistant génère une autorité de certification interne et un certificat par nœud, puis chaque composant est installé sur sa machine. L'ancien tout-en-un est conservé en snapshot comme point de retour.

```
# Sur l'indexer (.11) : décrire les nœuds puis générer les certificats
sudo bash ./wazuh-install.sh --generate-config-files
# → produit wazuh-install-files.tar (certificats + mots de passe)

# Installer chaque rôle sur sa VM
sudo bash ./wazuh-install.sh --wazuh-indexer node-1
sudo bash ./wazuh-install.sh --start-cluster
sudo bash ./wazuh-install.sh --wazuh-server wazuh-1      # sur .12
sudo bash ./wazuh-install.sh --wazuh-dashboard dashboard # sur .13
```

Les agents et le pare-feu sont ensuite re-raccordés au nouveau manager (.12) : la nouvelle autorité de certification impose un ré-enrôlement (changement d'adresse + redémarrage). Les trois agents repassent « Active ».

b) Corrélation personnalisée et processus suspect

Une règle de corrélation personnalisée (identifiant 100001) formalise la détection de force brute SSH soutenue : six échecs depuis une même source en moins de deux minutes lèvent une alerte de niveau 12, étiquetée MITRE ATT&CK T1110.

```
<group name="local,syslog,sshd,">
  <rule id="100001" level="12" frequency="6" timeframe="120">
    <if_matched_sid>5710</if_matched_sid>
    <same_source_ip />
    <description>Force brute SSH soutenue depuis $(srcip)</description>
    <mitre><id>T1110</id></mitre>
  </rule>
</group>
```

Un second cas d'usage détecte l'exécution d'un binaire depuis un répertoire temporaire (technique classique de malware) : les appels système execve sont journalisés par auditd sur l'endpoint, remontés à Wazuh, et une règle personnalisée (100020, MITRE T1059) lève une alerte de niveau 12. La mise au point des règles s'appuie sur l'outil wazuh-logtest, qui rejoue une ligne de journal et affiche la règle déclenchée.

c) Détection de vulnérabilités

Le module de détection de vulnérabilités compare l'inventaire logiciel des agents (collecté par Syscollector) à une base de CVE téléchargée depuis le service de threat intelligence de Wazuh, et stocke les résultats dans l'indexer. Après installation d'un paquet ancien et vulnérable, les CVE apparaissent dans le dashboard avec leur criticité.

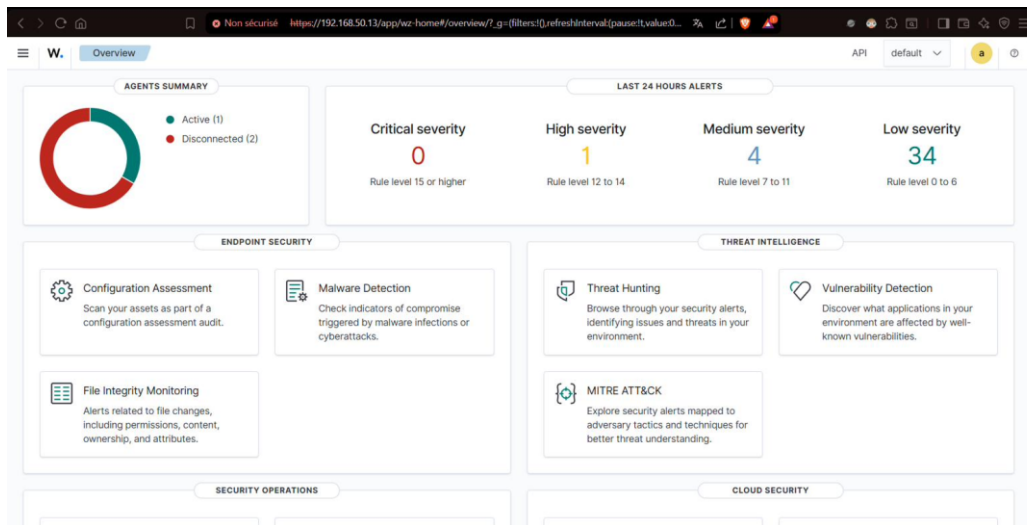


Figure 3 - Vue d'ensemble de la plateforme distribuée : résumé des agents et répartition des alertes des dernières 24 h par gravité, avec les modules Endpoint Security et Threat Intelligence.

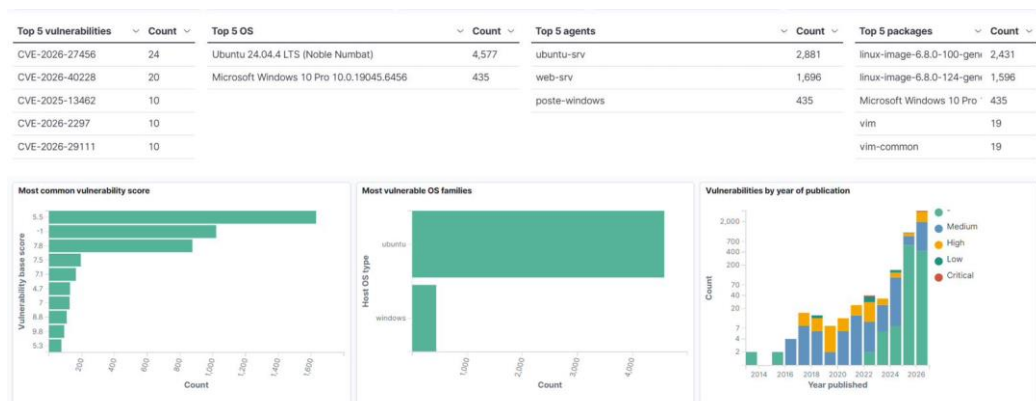


Figure 4 - Détection de vulnérabilités : top des CVE et des paquets affectés, score de sévérité et répartition des vulnérabilités par année de publication.

d) Alerting (e-mail + webhook)

Le moteur d'envoi intégré de Wazuh ne gère pas l'authentification SMTP moderne : on installe donc un relais local Postfix sur le manager, configuré en smarthost vers Gmail (port 587, mot de passe d'application), et Wazuh envoie vers localhost. Les alertes de niveau ≥ 10 déclenchent un e-mail. En parallèle, une intégration webhook envoie ces mêmes alertes vers un salon Discord.

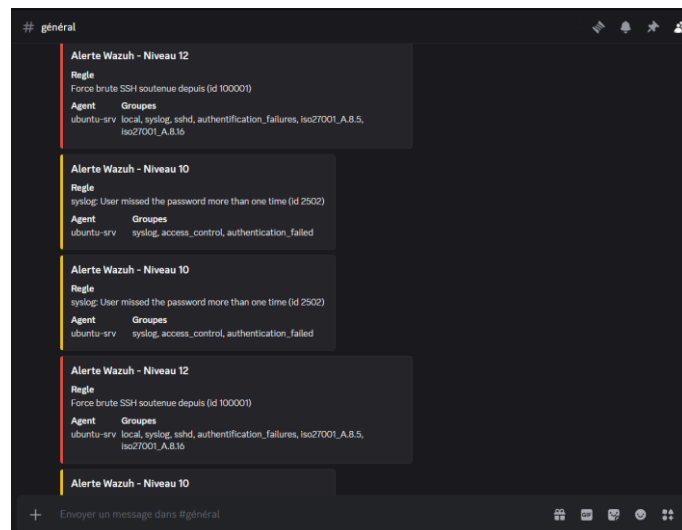


Figure 5 - Notification d'alerte reçue en temps réel : la chaîne d'alerting (e-mail via relais Postfix et webhook Discord) est fonctionnelle pour les alertes de gravité élevée.

e) Réention et conformité ISO 27001

Côté indexer, une politique de cycle de vie (ISM) supprime automatiquement les index d'alertes de plus de 90 jours ; côté manager, les fichiers d'alertes tournent quotidiennement. La conformité est étendue à l'ISO/IEC 27001:2022 via l'évaluation de configuration (SCA contre les référentiels CIS), l'étiquetage des règles avec des identifiants de contrôle ISO, et la génération de rapports PDF.

Enseignements de la Phase 2

Mémoire accentuée. Trois nœuds au lieu d'un saturent l'hôte : résolu par un fichier d'échange, l'allongement du délai de démarrage et la règle d'allumage sélectif.

Droits du kit de certificats. Le fichier wazuh-install-files.tar, généré en root, doit voir son propriétaire corrigé avant d'être copié vers les autres nœuds.

Bascule = ré-enrôlement. La nouvelle autorité de certification du manager impose que chaque agent se ré-enrôle, sans quoi il reste invisible.

Validation avant redémarrage. On valide toujours la configuration avec `wazuh-analysisd -t` avant de redémarrer le manager.

6.3 Phase 3 - Architecture d'entreprise (cluster, conteneurs, IDS, threat intel, réponse)

La phase 3 hisse la plateforme au niveau entreprise : résilience (cluster indexer avec réplication), conteneurisation (worker Wazuh + HAProxy sous Docker), IDS réseau (Suricata), renseignement sur les menaces et réponse automatisée. Toutes les briques ont été choisies pour leur faible empreinte mémoire afin de tenir dans les 16 Go.

a) Haute disponibilité : cluster indexer à deux nœuds

Un deuxième nœud indexer (192.168.50.15) est ajouté pour obtenir un vrai cluster OpenSearch avec réplication : chaque fragment de données (shard) est doublé sur l'autre nœud. Le fichier `opensearch.yml` de chaque nœud liste les deux membres (`discovery.seed_hosts`), et la réplication est activée sur les index Wazuh. Le cluster atteint l'état **GREEN** (64 shards dupliqués), et la coupure volontaire d'un nœud fait passer le cluster en « yellow » sans perte de service ni de données, preuve concrète de la tolérance à la panne.

Piège résolu - le config.yml de l'installateur

Le fichier `config.yml` embarqué dans `wazuh-install-files.tar` fait autorité pour l'installateur : il doit être mis à jour (le second nœud décommenté) **avant** de lancer l'installation de l'indexer-2. De plus, `discovery.seed_hosts` doit lister les deux adresses IP dans l'`opensearch.yml` du premier nœud.

b) Cluster de managers et conteneurisation

Un manager « worker » est lancé sous forme de conteneur Docker sur docker-host (192.168.50.14), à partir de l'image officielle wazuh/wazuh-manager:4.14.5, et rejoint le master (.12) grâce à une clé de cluster partagée. Le master synchronise automatiquement ses règles et décodeurs vers le worker.

```
# Sur docker-host : lancer le worker conteneurisé
cd ~/wazuh-worker && docker compose up -d

# Depuis le master (.12), vérifier que le worker a rejoint le cluster
sudo /var/ossec/bin/cluster_control -l
# → manager-master (master, .12) + manager-worker (worker, .14)
```

```
# Sur ubuntu-srv : lire le fichier EVE de Suricata via l'agent Wazuh
<localfile>
  <log_format>json</log_format>
  <location>/var/log/suricata/eve.json</location>
</localfile>
```

Piège critique résolu - décodeur JSON saturé

Chaque événement Suricata 8.x dépasse la limite par défaut de 256 champs du décodeur JSON de Wazuh (erreur « Too many fields for JSON decoder »). Correctif : porter `analysisd.decoder_order_size` à 1024 dans `local_internal_options.conf`, puis redémarrer le manager. La détection a ensuite été confirmée en conditions réelles.

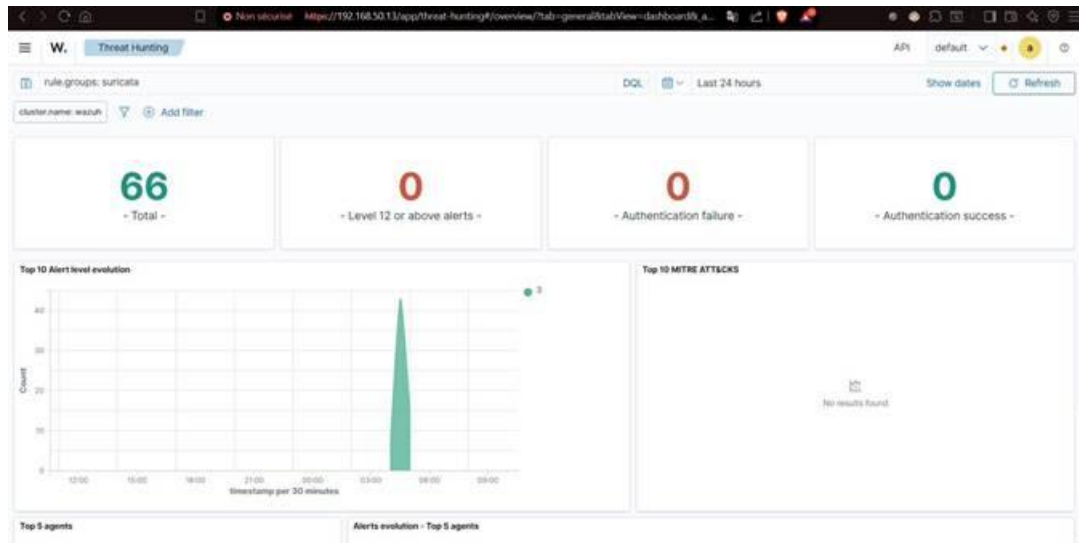


Figure 8 - Tableau de bord Threat Hunting filtré sur les alertes Suricata : les alertes de l'IDS réseau remontent et sont corrélées dans Wazuh (ici 66 alertes du groupe suricata sur 24 h).

e) Threat intelligence : listes d'IoC + VirusTotal

Deux mécanismes complémentaires ont été mis en place. D'abord, trois **listes CDB** d'indicateurs de compromission (IP, domaines, hachages) alimentées quotidiennement par des flux publics (Feodo Tracker, ET compromised IPs, ~636 IoC) via une tâche planifiée ; une IP malveillante déclenche alors une alerte dédiée (règle 99915). Ensuite, une intégration **VirusTotal** sur le FIM : à chaque modification d'un fichier surveillé, son hachage (jamais le fichier) est envoyé à l'API VirusTotal, qui renvoie le nombre de moteurs antivirus le considérant malveillant.

La chaîne a été prouvée de bout en bout avec un fichier de test EICAR (inoffensif mais universellement reconnu comme malveillant) : le FIM détecte sa création, VirusTotal l'analyse, et une alerte enrichie de niveau 12 (règle 87105, « 4 engines detected this file ») remonte, avec mapping MITRE T1203, références de conformité et lien VirusTotal. La notification Discord de niveau 12 a été reçue en temps réel.

f) Réponse automatisée aux incidents (Active Response)

L'Active Response fait passer la plateforme de la détection à la réaction : le module firewall-drop bloque automatiquement, au niveau du pare-feu local de la machine ciblée, l'adresse IP à l'origine d'une force brute SSH. Le déclencheur est rattaché à la règle de corrélation personnalisée 100001 (niveau 12), complétée par la règle native 5763 - un choix qui évite les faux positifs d'un blocage sur une simple tentative isolée et valorise le travail de corrélation des phases précédentes.

```
<active-response>
  <command>firewall-drop-lab</command>
  <location>local</location>      <!-- agir sur l'agent visé -->
  <rules_id>100001,5763</rules_id> <!-- force brute SSH -->
```

```
<timeout>600</timeout>      <!-- bannir 10 minutes -->
</active-response>
```

Démonstration de bout en bout (validée). Le scénario complet a été rejoué et vérifié : depuis Kali (192.168.50.105), une force brute SSH est lancée contre ubuntu-srv avec Hydra et la liste de mots de passe rockyou.txt. Au sixième échec en moins de deux minutes, la règle 100001 (niveau 12, MITRE T1110) se déclenche et invoque firewall-drop. Le journal de réponse (active-responses.log) montre la séquence firewall-drop: Starting → traitement de l'alerte → firewall-drop: Ended, et l'IP attaquante est effectivement bannie : deux règles DROP visant 192.168.50.105 apparaissent dans iptables. La seconde tentative de Hydra échoue alors avec « Timeout connecting to 192.168.50.30 », preuve que le trafic de l'attaquant est bien bloqué.

```
# Attaque depuis Kali
hydra -l fakeuser -P /usr/share/wordlists/rockyou.txt ssh://192.168.50.30 -t 4

# Sur ubuntu-srv : l'IP attaquante est bannie par firewall-drop
sudo iptables -L -n | grep 192.168.50.105
DROP 0 -- 192.168.50.105 0.0.0.0/0
DROP 0 -- 192.168.50.105 0.0.0.0/0
```

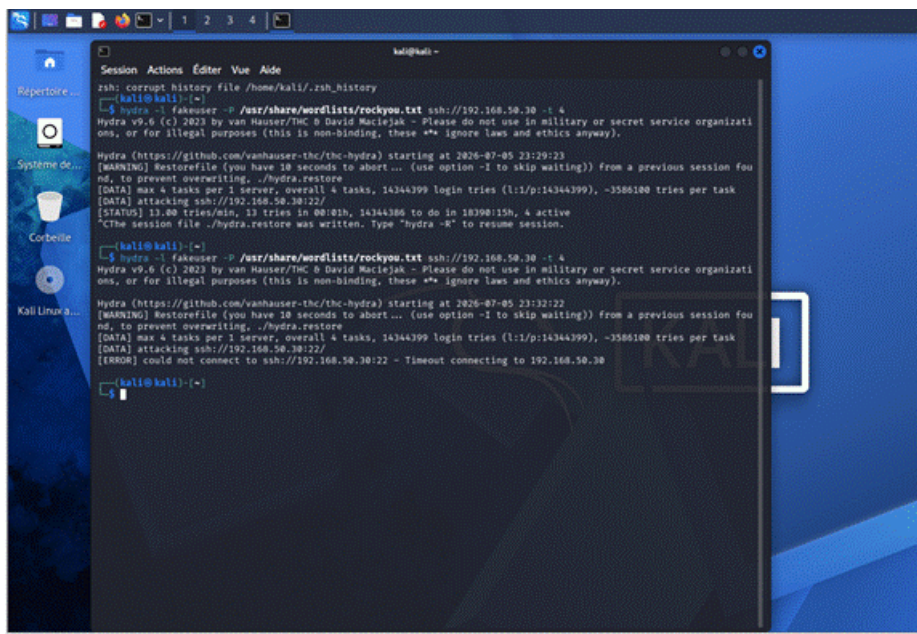


Figure 9 - Attaque de force brute SSH lancée depuis Kali avec Hydra contre ubuntu-srv (192.168.50.30). La seconde tentative se solde par un « Timeout connecting » : l'attaquant est déjà bloqué par l'Active Response.

```
wazuh@ubuntu-srv:~$ sudo tail -f /var/ossec/logs/active-responses.log
[sudo] password for wazuh:
2026/07/05 15:31:19 active-response/bin/restart.sh agent reload
2026/07/05 15:31:26 active-response/bin/restart.sh agent reload
2026/07/05 15:31:35 active-response/bin/restart.sh agent reload
2026/07/05 15:31:47 active-response/bin/restart.sh agent reload
2026/07/05 15:31:51 active-response/bin/restart.sh agent reload
2026/07/05 15:31:52 active-response/bin/restart.sh agent reload
2026/07/05 15:31:55 active-response/bin/restart.sh agent reload
2026/07/05 16:34:48 active-response/bin/restart.sh agent reload
2026/07/05 16:39:23 active-response/bin/restart.sh agent reload
2026/07/05 23:29:39 active-response/bin/firewall-drop: Starting
2026/07/05 23:29:39 active-response/bin/firewall-drop: [{"version":1,"origin":{"name":"manager-master","module":"wazuh-execd"},"command":"add","parameters":{"extra_args":{"id":"100001"},"alert":{"timestamp":"2026-07-05T23:29:38.067+0000","rule":{"level":12,"description":"Force brute SSH soutenue depuis ","id":"100001","mitre":{"id":"T1110"},"active":{"Credential Access"},"technique":{"Brute Force"},"frequency":5,"firedtimes":1,"wait":true,"groups":{"local":"syslog"},"authentication_failures":{"iso2001A.8.5"},"iso2001A.8.16"},"sdpr":{"TV.35.7.d"},"agent":{"id":"001","name":"ubuntu-srv","ip":"192.168.50.30"},"manager":{"name":"wazuh-manager"},"id":"1783294178.38629937"},"cluster":{"name":"wazuh","node":"manager-master"},"previous_output":{"Jul 05 23:29:37 ubuntu-srv sshd[2894]: Invalid user fakeuser from 192.168.50.105 port 49334\nJul 05 23:29:37 ubuntu-srv sshd[2897]: Invalid user fakeuser from 192.168.50.105 port 49300 [preauth]\nJul 05 23:29:37 ubuntu-srv sshd[2893]: Invalid user fakeuser from 192.168.50.105 port 49310\nJul 05 23:29:37 ubuntu-srv sshd[2891]: Invalid user fakeuser from 192.168.50.105 port 49300"},"full_log":{"Jul 05 23:29:37 ubuntu-srv sshd[2895]: Invalid user fakeuser from 192.168.50.105 port 49338"},"predecoder":{"program_name":"sshd","timestamp":"Jul 05 23:29:37","hostname":"ubuntu-srv"},"decoder":{"parent":"sshd","name":"sshd"},"data":{"srcip":"192.168.50.105","srcport":"49338","srcuser":"fakeuser"},"location":"Journal"},"program":"active-response/bin/firewall-drop"}]
2026/07/05 23:29:39 active-response/bin/firewall-drop: [{"version":1,"origin":{"name":"firewall-drop","module":"active-response"},"command":"check_keys","parameters":{"keys":["192.168.50.105"]}]
2026/07/05 23:29:39 active-response/bin/firewall-drop: [{"version":1,"origin":{"name":"manager-master","module":"wazuh-execd"},"command":"continue","parameters":{"extra_args":{"id":"100001"},"alert":{"timestamp":"2026-07-05T23:29:38.067+0000","rule":{"level":12,"description":"Force brute SSH soutenue depuis ","id":"100001","mitre":{"id":"T1110"},"active":{"Credential Access"},"technique":{"Brute Force"},"frequency":5,"firedtimes":1,"wait":true,"groups":{"local":"syslog"},"authentication_failures":{"iso2001A.8.5"},"iso2001A.8.16"},"sdpr":{"TV.35.7.d"},"agent":{"id":"001","name":"ubuntu-srv","ip":"192.168.50.30"},"manager":{"name":"wazuh-manager"},"id":"1783294178.38629937"},"cluster":{"name":"wazuh","node":"manager-master"},"previous_output":{"Jul 05 23:29:37 ubuntu-srv sshd[2894]: Invalid user fakeuser from 192.168.50.105 port 49334\nJul 05 23:29:37 ubuntu-srv sshd[2897]: Invalid user fakeuser from 192.168.50.105 port 49300 [preauth]\nJul 05 23:29:37 ubuntu-srv sshd[2893]: Invalid user fakeuser from 192.168.50.105 port 49310\nJul 05 23:29:37 ubuntu-srv sshd[2891]: Invalid user fakeuser from 192.168.50.105 port 49300"},"full_log":{"Jul 05 23:29:37 ubuntu-srv sshd[2895]: Invalid user fakeuser from 192.168.50.105 port 49338"},"predecoder":{"program_name":"sshd","timestamp":"Jul 05 23:29:37","hostname":"ubuntu-srv"},"decoder":{"parent":"sshd","name":"sshd"},"data":{"srcip":"192.168.50.105","srcport":"49338","srcuser":"fakeuser"},"location":"Journal"},"program":"active-response/bin/firewall-drop"}]
2026/07/05 23:29:40 active-response/bin/firewall-drop: Ended
wazuh@ubuntu-srv:~$ sudo iptables -L -n | grep 192.168.50.105
DROP    0    --  192.168.50.105          0.0.0.0/0
DROP    0    --  192.168.50.105          0.0.0.0/0
wazuh@ubuntu-srv:~$
```

Figure 10 - Preuve du blocage : le journal active-responses.log montre firewall-drop qui traite l'alerte de niveau 12 (règle 100001, force brute SSH depuis 192.168.50.105), et iptables confirme deux règles DROP sur l'IP attaquante.

Active Response - pleinement fonctionnel

Le mécanisme est **validé de bout en bout** : détection par corrélation (règle 100001, niveau 12), déclenchement automatique de firewall-drop, et blocage effectif de l'IP attaquante dans iptables (règles DROP visibles, attaque suivante en timeout). La configuration a été répliquée à l'identique sur le master et le worker, car les blocs active-response ne sont pas synchronisés par le cluster, contrairement aux règles et décodeurs. La plateforme ne se contente donc plus de détecter : elle **répond** automatiquement aux attaques, l'aboutissement de la logique XDR du projet.

g) Vue d'ensemble de la plateforme

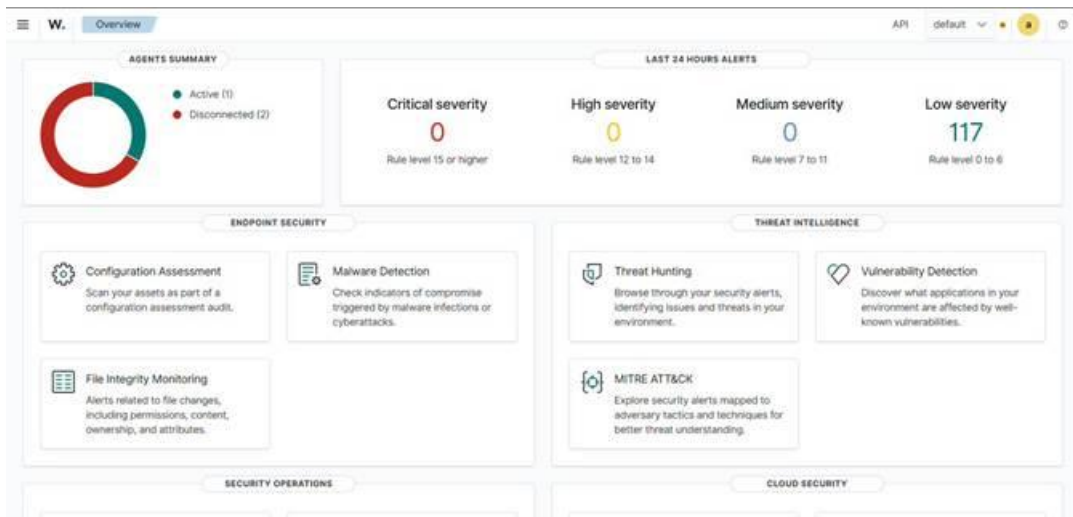


Figure 11 - Vue d'ensemble (Overview) de la plateforme d'entreprise : résumé des agents, alertes par gravité et modules Endpoint Security, Threat Intelligence (Threat Hunting, Vulnerability Detection, MITRE ATT&CK).

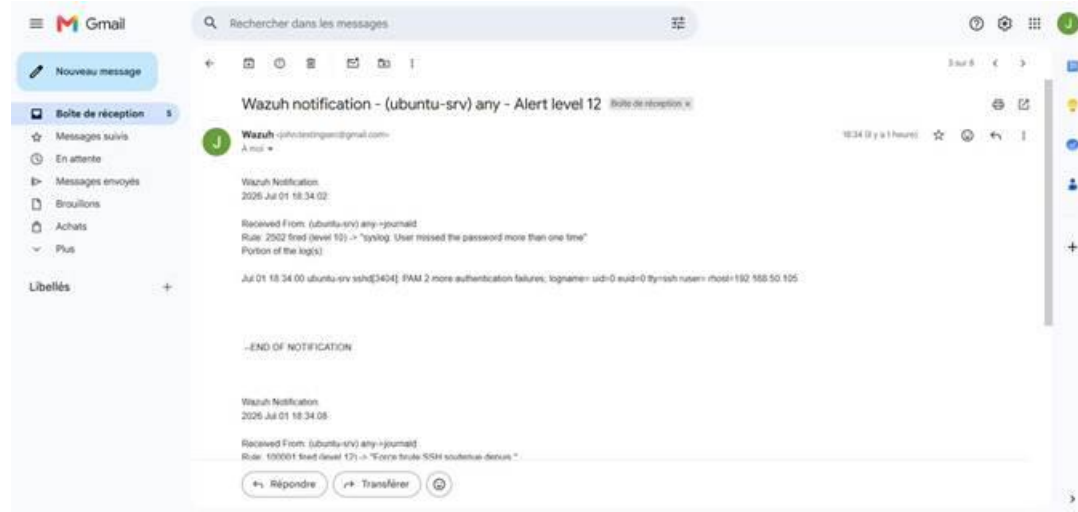


Figure 12 - Notification e-mail d'une alerte de niveau 12 (force brute SSH soutenue, règle 100001) reçue dans la boîte de réception : la chaîne d'alerting fonctionne sur l'architecture finale.

7. Suite de tests de recette

Chaque exigence a été validée par un ou plusieurs tests reproductibles, inspirés des cas d'usage du cahier des charges. Le numéro de test est unique. La colonne « Preuve » renvoie à la figure ou au journal qui atteste du résultat ; la colonne OK/NOK est laissée à la disposition du jury externe pour la validation en soutenance. Deux scénarios nominaux sont d'abord détaillés intégralement, puis l'ensemble des tests est présenté en tableaux.

7.0 Scénarios nominaux détaillés

Scénario nominal A - Détection et corrélation d'une force brute SSH

Ce scénario illustre la valeur ajoutée du SIEM : transformer une série d'événements isolés en une alerte qualifiée de haute gravité, puis la notifier.

- **Pré-requis** : groupe « Détection » allumé (socle + ubuntu-srv + Kali) ; agent ubuntu-srv actif ; règle personnalisée 100001 chargée sur le manager ; intégration Discord active.
- **Étape 1 - Attaque**. Depuis Kali, on lance une force brute SSH soutenue contre ubuntu-srv (par exemple avec Hydra ou une boucle de connexions échouées vers un utilisateur inexistant).
- **Étape 2 - Détection unitaire**. Le manager décode chaque échec et lève la règle native 5710 (niveau 5, « Attempt to login using a non-existent user »).
- **Étape 3 - Corrélation**. Au sixième échec de la même source en moins de 120 s, la règle personnalisée 100001 (niveau 12, « Force brute SSH soutenue », MITRE T1110) se déclenche, étiquetée iso27001_A.8.5 et A.8.16.
- **Étape 4 - Notification**. L'alerte de niveau 12 est poussée en temps réel vers le salon Discord et par e-mail (relais Postfix → Gmail).
- **Résultat attendu** : apparition de la règle 100001 (niveau 12) dans le dashboard et dans alerts.log, et réception de la notification. **Résultat obtenu** : conforme (cf. Figures 5 et 11).

Scénario nominal B - Threat intelligence : fichier malveillant détecté par VirusTotal

Ce scénario illustre l'enrichissement d'une alerte de surveillance d'intégrité par du renseignement externe.

- **Pré-requis** : FIM en temps réel sur un dossier surveillé ; intégration VirusTotal active (clé API) sur le manager.
- **Étape 1 - Dépôt**. On dépose un fichier de test EICAR (inoffensif mais universellement reconnu comme malveillant) dans le dossier surveillé de ubuntu-srv.
- **Étape 2 - FIM**. Le module syscheck détecte la création du fichier et déclenche l'intégration VirusTotal, qui envoie le hachage (jamais le fichier) à l'API.
- **Étape 3 - Enrichissement**. VirusTotal renvoie un verdict ; Wazuh lève la règle 87105 (niveau 12, « 4 engines detected this file »), avec mapping MITRE T1203 et lien VirusTotal.
- **Résultat attendu** : alerte 87105 (niveau 12) enrichie du verdict et notification Discord de niveau 12. **Résultat obtenu** : conforme.

7.1 Exigence 1 - Collecte centralisée et supervision (Phase 1)

Objectif : centraliser les journaux d'au moins quatre sources hétérogènes et offrir un tableau de bord de supervision (réf. cahier des charges, fonctions 1 et 4).

N°	Description	Scénario d'exécution	Résultat attendu	Preuve	OK/NOK
1	Accès au dashboard	Connexion HTTPS admin à https://.10 (puis .13 en Ph. 2-3)	Interface accessible, connexion réussie	Fig. 10	OK

N°	Description	Scénario d'exécution	Résultat attendu	Preuve	OK/NOK
2	Agents actifs	Enrôler Windows, Linux, web ; consulter l'inventaire	3 agents au statut « Active »	Fig. 2	OK
3	Source agentless	pfSense émet du syslog vers le manager (514/UDP)	Événements pfSense visibles dans Discover	tcpdump 514	OK
4	Tableau de bord	Consulter la vue de synthèse par gravité	Répartition des alertes (629 événements)	Fig. 3, 10	OK

7.2 Exigence 2 - Détection et corrélation (Phases 1-2)

Objectif : détecter des comportements suspects et les corréler en alertes qualifiées (fonction 2 ; cas d'usage force brute et processus suspect). Voir le scénario nominal A.

N°	Description	Scénario d'exécution	Résultat attendu	Preuve	OK/NOK
5	Force brute SSH (natif)	Échecs SSH répétés depuis Kali	Règle 5710 puis 5712 (niveau 10)	alerts.log	OK
6	Force brute SSH (custom)	Force brute soutenue (Hydra) - scénario A	Règle 100001 (niveau 12, MITRE T1110)	Fig. 5, 11	OK
7	Processus suspect	Exécution d'un binaire depuis /tmp	Règle 100020 (niveau 12, MITRE T1059)	alerts.log	OK
8	Scan web (bonus)	gobuster depuis Kali vers web-srv	Règles 31101 puis 31151 (niveau 10)	alerts.log	OK

7.3 Exigence 3 - Intégrité des fichiers (FIM, Phase 1)

Objectif : alerter en temps réel sur toute création, modification ou suppression d'un fichier sensible (fonction 3).

N°	Description	Scénario d'exécution	Résultat attendu	Preuve	OK/NOK
9	Suppression fichier /etc	rm sur un fichier surveillé de /etc	Règle 553 (niveau 7) + tags RGD/PCI	alerts.log	OK
10	Modification fichier	echo append sur un fichier surveillé	Alerte FIM temps réel (règle 550)	alerts.log	OK

7.4 Exigence 4 - Vulnérabilités, alerting, rétention, conformité (Phase 2)

N°	Description	Scénario d'exécution	Résultat attendu	Preuve	OK/NOK
11	Détection de vulnérabilités	Installer un paquet ancien vulnérable	CVE affichée dans Vulnerability Detection	Fig. 4	OK
12	Alerte e-mail	Générer une alerte ≥ niveau 10	E-mail reçu (relais Postfix → Gmail)	Fig. 11	OK
13	Alerte webhook	Générer une alerte ≥ niveau 10	Message reçu dans le salon Discord	Fig. 5	OK
14	Rétention ISM	Vérifier la politique d'index	Politique 90 j appliquée aux wazuh-alerts-*	Dashboard ISM	Ok
15	Conformité ISO 27001	Consulter SCA/CIS et tags des règles	Vue SCA + tags ISO + rapport généré	Dashboard SCA	OK

7.5 Exigence 5 - Cluster, conteneurs, IDS, threat intel, réponse (Phase 3)

N°	Description	Scénario d'exécution	Résultat attendu	Preuve	OK/NOK
16	Cluster indexer HA	Former le cluster 2 nœuds	Cluster « GREEN », 64 shards dupliqués	_cat/nodes	OK

N°	Description	Scénario d'exécution	Résultat attendu	Preuve	OK/NOK
17	Bascule / tolérance de panne	Arrêter le nœud indexer-2	Cluster « yellow », service maintenu	_cluster/health	OK
18	Worker conteneurisé	docker compose up + cluster_control	master + worker listés dans le cluster	Fig. 6	OK
19	Équilibrage HAProxy	Pointer les agents vers le VIP .14	Agents répartis master/worker	Fig. 6, 7	OK
20	IDS Suricata	Scan Nmap depuis Kali vers ubuntu-srv	Alertes du groupe « suricata » dans Wazuh	Fig. 8	OK
21	Threat intel - IoC	IP présente dans la liste CDB	Règle 99915 déclenchée (niveau 9)	alerts.log	OK
22	Threat intel - VirusTotal	Déposer un fichier EICAR - scénario B	Règle 87105 (niveau 12), verdict VirusTotal	alerts.json	OK
23	Active Response (armé)	Config + files de réponse	wazuh-analysisd -t = Exit 0, AR connectée	Fig. 9, 10	OK

Synthèse de la recette

L'ensemble des exigences des phases 1 et 2 est validé. En phase 3, les capacités de cluster/HA, conteneurisation, équilibrage de charge, IDS, threat intelligence et l'Active Response (blocage automatique de l'IP attaquante démontré de bout en bout). Le périmètre non réalisé est détaillé en section 8.

8. Périmètre non réalisé et travail restant

Par souci de transparence et pour ne présenter que ce qui a été effectivement livré, cette section recense les exigences avancées de la phase 3 qui n'ont pas été réalisées, avec la raison et la piste de mise en œuvre. Ces éléments figuraient au plan de réalisation cible mais n'ont pas pu être menés à leur terme, essentiellement en raison de la contrainte matérielle des 16 Go et du temps disponible.

Exigence (Phase 3)	État	Raison	Piste de réalisation
Gestion des identités - Active Directory (Samba AD)	X Non réalisé	Cohabitation mémoire difficile avec le reste sur 16 Go	Samba AD sur petite VM, ou Windows Server sur hôte 32 Go+
Intégration cloud - AWS CloudTrail	X Non réalisé	Non mise en place (compte / temps)	Module aws-s3 natif de Wazuh, sans coût de RAM local
Analyse comportementale étendue (command monitoring)	X Non réalisé	Priorisée après les briques structurantes	Ajout de sondes (ports en écoute, rootcheck) sur les agents
Conformité PCI DSS	X Non réalisé	Repose en partie sur AD et le comportemental non faits	Étiquetage PCI natif + SCA, à activer une fois le socle complet

Il est important de noter que ces manques ne remettent pas en cause l'atteinte de l'objectif central du projet : livrer une plateforme SIEM/XDR open-source, distribuée, conteneurisée, résiliente et conforme au RGPD et à l'ISO 27001. Les briques d'entreprise les plus structurantes (cluster à réplication, conteneurs, équilibrage de charge, IDS, threat intelligence) sont bien en place et validées. Le périmètre non réalisé constitue une feuille de route d'industrialisation cohérente avec la logique progressive du projet.

9. Conclusions et perspectives

Dans cette étude, nous avons démontré qu'une PME peut se doter, sans aucun coût de licence, d'une plateforme de supervision de sécurité de niveau professionnel. Le projet OSIRIS a conduit une trajectoire complète, du besoin exprimé par le RSSI jusqu'à une architecture d'entreprise : d'un déploiement tout-en-un supervisant quatre sources hétérogènes, la plateforme est devenue une architecture distribuée enrichie de corrélation, de détection de vulnérabilités et d'alerting, puis une architecture résiliente et conteneurisée dotée d'un cluster à réplication, d'un équilibrage de charge, d'un IDS réseau, de renseignement sur les menaces et d'une réponse automatisée. Le SIEM/XDR passe ainsi d'un outil qui observe à une plateforme qui détecte, corrèle et répond, l'objectif même d'un XDR d'entreprise.

9.1 Bilan et compétences acquises

Le projet a permis d'articuler des compétences techniques (architecture, réseau, sécurité, virtualisation, conteneurisation) et des compétences de gestion de projet (cadrage, planification, gouvernance, arbitrages coût-délai-qualité, gestion des risques). L'incident de saturation mémoire de la phase 1, arbitré par un re-séquencement plutôt qu'une dégradation de la qualité, illustre concrètement la capacité d'adaptation attendue d'une équipe projet. Le choix d'une solution unique et évolutive (Wazuh) et le découpage en phases à recettes fermes se sont révélés être les principaux facteurs de succès.

9.2 Analyse critique et limites

Les limites assumées relèvent surtout de l'environnement de laboratoire : un hôte de 16 Go impose une démonstration séquentielle (groupes d'allumage) et un cluster à deux nœuds indexer présente un risque théorique de split-brain. Certaines exigences avancées (Active Directory, cloud AWS, analyse comportementale étendue, PCI DSS) n'ont pas été réalisées. Ces limites tracent la feuille de route d'une industrialisation, sans remettre en cause la validité de la démonstration.

Point	Limite dans le laboratoire	Amélioration en production
Quorum du cluster indexer	2 nœuds : risque théorique de split-brain	3e nœud (ou nœud voting-only) pour un quorum impair
Conteneurisation	Docker Compose sur une seule VM	Orchestration Kubernetes / K3s multi-hôtes
Haute disponibilité manager	1 master + 1 worker	Plusieurs workers + master redondé
Démonstration	Séquentielle (groupes d'allumage)	Hôte 32 Go+ : toutes les VM simultanément
Périmètre avancé	AD, cloud, PCI DSS non réalisés	Ajout progressif selon budget et besoins

9.3 Perspectives

- **Compléter le périmètre.** Intégrer l'Active Directory (gestion des identités), le cloud (AWS CloudTrail), l'analyse comportementale étendue et la conformité PCI DSS, en réutilisant les modules natifs de Wazuh.
- **Industrialiser.** Passer d'un hôte unique à une infrastructure plus puissante (32 Go+, ou cloud) pour lever la contrainte des groupes d'allumage ; orchestrer les conteneurs avec Kubernetes/K3s ; ajouter un troisième nœud indexer pour un quorum robuste.
- **Exploiter.** Transférer la plateforme à l'exploitation de la PME (SOC / administrateur) : réglage fin des règles pour réduire les faux positifs, alimentation automatique des listes d'IoC, revue périodique de la rétention et des accès.

Mot de la fin

OSIRIS confirme la faisabilité économique et technique d'une supervision de sécurité open-source pour une PME : un socle conforme au RGPD dès le premier jour, une montée en maturité maîtrisée sans changement d'outil, et une plateforme qui, au terme du projet, ne se contente plus de journaliser mais détecte, corrèle et répond aux menaces.

Annexe A - Plan d'adressage et flux réseau

A.1 Plan d'adressage complet (VMnet3 - 192.168.50.0/24)

Machine	Adresse	Rôle	Phase
pfSense	.1	Passerelle / pare-feu / source syslog	1
dc-srv	.10 (réservée)	Contrôleur de domaine (non réalisé)	3
wazuh-indexer	.11	Indexer - nœud 1 du cluster	2
wazuh-manager	.12	Manager master + API	2
wazuh-dashboard	.13	Dashboard (visualisation)	2
docker-host	.14	Worker Wazuh + HAProxy (conteneurs)	3
wazuh-indexer-2	.15	Indexer - nœud 2 (réplication)	3
poste-windows	.20	Endpoint Windows (agent)	1
ubuntu-srv	.30	Endpoint Linux + Suricata (IDS)	1 → 3
web-srv	.40	Serveur web Apache (agent)	1
Kali	~.105 (DHCP)	Machine d'attaque (tests)	1

A.2 Flux et ports principaux

Flux	Port / Protocole	Usage
Agent → manager (via HAProxy)	1514/TCP (TLS)	Remontée chiffrée des événements
Agent → master	1515/TCP (TLS)	Enrôlement initial (échange de clé)
Manager → indexer	9200/TCP (HTTPS)	Acheminement des alertes (Filebeat)
Indexer ↔ indexer	9300/TCP	Transport interne du cluster
Master ↔ worker	1516/TCP	Synchronisation du cluster de managers
pfSense → manager	514/UDP	Syslog agentless
Analyste → dashboard	443/TCP (HTTPS)	Interface web

Annexe B - Bonnes pratiques et pièges consolidés

Enseignements transverses qui font désormais partie des réflexes du projet et facilitent sa reprise :

Difficulté	Résolution / leçon
Droits de client.keys	authd (root) réinitialise les droits au redémarrage : chown wazuh:wazuh (override systemd pour l'automatiser)
config.yml de l'installateur	Le fichier interne à l'archive fait autorité : le mettre à jour avant d'installer un nouveau nœud
Décodeur JSON Suricata	Relever decoder_order_size à 1024 pour absorber les événements Suricata 8.x
Config Active Response non clusterisée	Répliquer les blocs command / active-response sur master et worker
Rattachement mouvant HAProxy	Un agent peut basculer master↔worker : fixer temporairement le nœud pour une démo déterministe
Validation avant redémarrage	Toujours lancer wazuh-analysisd -t (et non configtest) avant de redémarrer le manager
Mémoire de l'hôte	Groupes d'allumage stricts + snapshots avant toute opération sensible

Annexe C - Intitulés des sections (rapport en français)

La trame Solution Factory prévoit une annexe rappelant les intitulés de sections en français pour un projet mené dans cette langue. Le présent rapport étant intégralement rédigé en français, cette correspondance est fournie ci-dessous à titre de repère par rapport au plan-type anglais de la trame.

Plan-type (trame)	Intitulé retenu dans ce rapport
Introduction	1. Introduction
Documentation and terminology	2. Documentation et terminologie
Reference documents / Glossary (Terms, Acronyms)	2.1 Documents de référence / 2.2 Glossaire (Termes, Acronymes)
Project Description	3. Description du projet
Function and contribution of the actors	4. Fonctions et contribution des acteurs
Technical feasibility studies	5. Études de faisabilité et environnement de test
Physical / Hardware / Software environment, Data	5.1 Physique / 5.2 Matériel / 5.3 Logiciel / 5.4 Données
Test suite (Requirements)	6. Suite de tests de recette (Exigences)
Conclusions & Perspectives	8. Conclusions et perspectives
Appendices	Annexes A, B, C

Une section supplémentaire, non prévue au plan-type mais ajoutée par souci de transparence, complète le rapport : la section 8 « Périmètre non réalisé et travail restant », qui distingue clairement ce qui a été livré de ce qui reste à faire.

Fin du document

Rapport final du projet OSIRIS - Groupe SR02 / SR-2E - EFREI Paris. Ce document présente l'intégralité du périmètre réalisé et identifie honnêtement le travail restant.